

Assignment 1 LLM Transcript – Brandon Chanderban

In terms of selecting the dataset to be utilized, this portion was done without the intervention of any LLM (in this case, OpenAI's ChatGPT, which was used elsewhere in the project scope). As stated within the Q Markdown file, the Breast Cancer Wisconsin Dataset was selected in part due to my coming across an article about recent advancements in pancreatic cancer treatment.

Some of the initial R code, such as subsetting and selecting variables, was developed using knowledge gained from completing courses on the DataCamp platform. In cases where I required specific actions to be performed on the dataframe and did not have sufficient working knowledge of how to implement them, I consulted ChatGPT. I obtained responses, followed up to clarify the logic (if not already explained), and then incorporated the solution into my code.

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Within my script to read my raw data (within a variable named 'url') into a dataframe df1, how do I ensure that all missing data (encoded as "?") are realized as missing values?

To ensure that all missing values encoded as "?" are correctly interpreted as NA when reading your raw data from the URL into the df1 dataframe, you can use the na.strings argument within the read.csv() function as shown below:

```
df1 <- read.csv(file = url, na.strings = "?")
```

This tells R to treat all instances of "?" in the dataset as NA, which is R's standard representation for missing values.

How may I then remove these records containing missing entries, and then determine the number of removed rows?

You can remove records containing missing entries using the na.omit() function and assign the cleaned result to a new dataframe (e.g., df1_clean). To determine how many rows were removed in the cleaning process, you can subtract the number of rows in the cleaned dataframe from the original. Here's the code to do that:

```
df1_clean <- na.omit(df1)
n_dropped <- nrow(df1) - nrow(df1_clean)
```

```
cat("Number of rows removed due to missing values:", n_dropped)
```

This code performs the following:

- `na.omit(df1)` removes all rows with any NA values.
- `nrow()` gives the number of rows before and after cleaning.
- `cat()` prints out the number of removed rows for easy interpretation.

How may I recode the variables within the target 'Class', replacing instances of 2 with "benign" and 4 with "malignant"?

You can use the `recode_factor()` function from the **dplyr** package to recode the values in the class column. Here's how to do it:

```
df1_clean <- df1_clean %>%  
  mutate(class = recode_factor(class, `2` = "benign", `4` = "malignant"))
```

This code:

- Uses the `mutate()` function to transform the class column.
- `recode_factor()` replaces the numeric values: 2 becomes "benign" and 4 becomes "malignant".
- The result is a factor variable with meaningful labels, which is helpful for modeling and interpretation.

Of the dataset's 10 features, which six would be the most effective/accurate in predicting the cancer class (benign or malignant) of breast tissue?

Great question — and a really important one for building a meaningful model or analysis!

To determine which of the 10 predictors in the Breast Cancer Wisconsin dataset are most effective for predicting class (benign vs malignant), we can look at correlations, feature importance from models, or review domain knowledge and prior studies.



The 10 Variables in the Dataset (excluding `sample_code_number` and `class`):

```
clump_thickness  
uniformity_cell_size  
uniformity_cell_shape
```

marginal_adhesion
single_epithelial_cell_size
bare_nuclei
bland_chromatin
normal_nucleoli
mitoses

✅ Top 6 Predictors (based on prior studies, correlation, and practical significance):

Based on domain literature, public analyses (like those on Kaggle), and exploratory data analysis, the top six predictive features are usually:

uniformity_cell_size
uniformity_cell_shape
bare_nuclei
clump_thickness
bland_chromatin
normal_nucleoli

 Why these?

uniformity_cell_size and uniformity_cell_shape are consistently shown to be highly correlated with malignancy — cancerous cells often have irregular shapes and sizes. bare_nuclei is a powerful indicator — many malignant cells have exposed nuclei. clump_thickness reflects cell crowding — often elevated in tumors. bland_chromatin and normal_nucleoli are tied to the nuclear appearance — a key indicator of abnormality.

How may I utilize the gt library to display a clean table of the first 10 entries within the dataframe named df_model?

You can use the gt package to create a neatly formatted table by piping your dataframe into gt() after selecting the top rows. Here's the code:

```
df_model %>%  
  slice_head(n = 10) %>%  
  gt() %>%  
  tab_header(  
    title = "Sample of Cleaned and Subsetted Dataset"  
  )
```

This code:

- Uses `slice_head(n = 10)` to select the first 10 rows of `df_model`.
- Pipes the result into `gt()` to generate a formatted table.
- Adds a title to the table using `tab_header()`, which improves readability in your output document or report.

What are some avenues of exploratory data analysis that may be conducted with the Breast Cancer Wisconsin dataset?

Some useful avenues of exploratory data analysis (EDA) for the Breast Cancer Wisconsin dataset include:

1. **Summary Statistics**

Generate descriptive statistics (mean, median, min, max, standard deviation) for the numeric features to understand their distribution and scale.

```
summary(df_model)
```

2. **Class Distribution**

Examine the proportion of benign vs malignant cases to assess class balance.

```
table(df_model$class)
```

3. **Correlation Matrix**

Create a correlation matrix to identify which features are most strongly related to one another. This can help in feature selection and understanding multicollinearity.

```
cor_matrix <- cor(select(df_model, -class))
```

```
corrplot(cor_matrix, method = "color", tl.cex = 0.8)
```

4. **Boxplots or Violin Plots**

Visualize the distribution of features across the two classes to identify patterns.

```
library(ggplot2)
```

```
ggplot(df_model, aes(x = class, y = uniformity_cell_size)) +  
  geom_boxplot()
```

5. **Pair Plots**

Use `GGally::ggpairs()` to show relationships between multiple variables, separated by class.

6. **Histograms**

Plot histograms for each numeric variable to assess skewness and modality.

These EDA steps can help uncover structure in the data, guide preprocessing decisions, and inform the design of predictive models.

`cor_matrix <- cor(select(df_model, -class)) corrplot(cor_matrix, method = "color", tl.cex = 0.8)` -- What does the 'tl.cex' section of the code establish, and what does the plot itself explain? The correlation between matching pairs of the cancer features?

Yes, exactly.

The `tl.cex` argument in the `corrplot()` function controls the **text label size** for the variable names in the correlation matrix plot. Adjusting this value helps with readability — for example, `tl.cex = 0.8` sets the text size to 80% of the default.

The plot itself is a **correlation matrix visualization** that shows the **pairwise Pearson correlation coefficients** between all numeric variables in your dataframe (in this case, the cancer features, excluding the target class).

- Values close to **+1 (dark blue)** indicate a strong **positive linear correlation** between two features — they tend to increase together.
- Values near **-1 (dark red)** indicate a strong **negative linear correlation** — as one increases, the other decreases.
- Values around **0** suggest **little to no linear relationship**.

This visual tool helps quickly identify which features are closely related and could be redundant, or which may be particularly informative when predicting cancer class.